

Introduction to Computer Programming

Prof. Giorgio Piccardo

`gpiccardo@luiss.it`

Department of Business and Management
Viale Romania 32, 00197 Rome



A.Y. 2024-2025

Outline

- 1 Introduction to Strings
- 2 Slicing
- 3 Operations on Strings
- 4 Iterating over Strings
- 5 Changing Strings
- 6 Build-in String Methods
 - `str.capitalize()`
 - `str.lower()` and `str.upper()`
 - `str.count()`
 - `str.endswith()` and `str.startswith()`
 - `str.find()` and `str.index()`
 - `str.is*()`
- 7 References
- 8 Acknowledgments

Introduction to Strings

Recall from Lecture 4 that:

- Strings are objects that can host letters, numbers, spaces, special characters.

More in detail:

- strings are among the most popular types in Python
- we can create strings simply by enclosing characters in quotes
- Python treats single quotes as the same as double quotes¹
- to create a string, just perform a variable assignment

Examples:

```
var1 = 'Hello World!'
```

```
var2 = "Python is cool! (hopefully)"
```

¹ Caveat: if your string contains a quote (e.g., `let's dance`), enclose the string in double quotes. Similarly, if your string contains double quotes (e.g., `I say "hello"`), enclose the string in single quotes.

- Python **does not support a character type**: chars are treated as strings of length 1, thus also considered a substring
- To access substrings, use the **square brackets for slicing** along with the **index or indices** to obtain your substring

Definition

The slice is a segment of the string. It can be gathered as `string_name[start:stop]`.

Note

Note that we include **start** in the slice, but we only go up to (and **not including**) **stop**. Further, indexing is **0-based** (the 1st element has index 0, the 2nd element has index 1 and so on ...).

Examples:

```

var1 = 'Hello World!'
var2 = "Python is cool! (hopefully)"
print("var1[0] = ", var1[0])           # indexing
print("var2[1:5] = ", var2[1:5])       # slicing
  
```

Slicing

Syntax of slicing:

```
s[start:stop]    # chars start through stop-1
s[start:]        # chars start through the rest of the string
s[:stop]         # chars from the beginning through stop-1
s[:]             # a full copy of the string
```

There is also the **step** value, which can be used with any of the above:

```
s[start:stop:step]  # start through not past stop, by step
```

start and **stop** can be negative numbers: this means that Python starts counting from the end of the string

```
s[-1]    # last char in the string
s[-2:]    # last two chars in the string
s[:-2]    # everything except the last two chars
```


Operations on Strings

Recall from Lecture 4 the two operators:

Concatenate: with the `+` operator

Replicate: with the `*` operator

Examples:

```
>>> "Hello " + "World!"    # yields "Hello World!"  
>>> "Python"*3             # yields "PythonPythonPython"
```

More operations on strings. Let `a = "Hello"` and `b = Python`.

Operator	Description	Example
<code>+</code>	Concatenation: adds values on either side of the operator	<code>a+b</code> yields <code>"HelloPython"</code>
<code>*</code>	Replication: creates new strings concatenating multiple copies of the original string	<code>a*2</code> yields <code>"HelloHello"</code>
<code>[]</code>	Slice: gives the character from the given index	<code>a[1]</code> yields <code>"e"</code>
<code>[:]</code>	Range: gives the character(s) from the given range	<code>a[1:4]</code> yields <code>"ell"</code>
<code>==</code>	Equality check: yields <code>True</code> if two strings are equal	<code>"Hello" == a</code> yields <code>True</code>
<code>in</code>	Membership: yields <code>True</code> if a substring appears in the string	<code>"He" in a</code> yields <code>True</code>
<code>not in</code>	Membership: yields <code>True</code> if a substring does not appear in the string	<code>"Me" not in a</code> yields <code>True</code>

Iterating over Strings

You can iterate over strings (i.e., get one character at the time) via **for** or **while** loops:

```
s = "Python Program"
for letter in s:
    print(letter)
```

```
s = "Python Program"
i = 0
while i < len(s):
    print(s[i])
    i = i + 1
```

Note

`len(s)` is a built-in function that returns the length of the string `s`.

Example: count the number of times "a" appears in s

```

s = "Python Program"
count = 0
for letter in s:
    if letter == 'a':
        count = count + 1
print(count)                # prints 1
    
```

Example: count the number of times "p" appears in s

```

s = "Python Program"
count = 0
i = 0
while i < len(s):
    if s[i] == 'p':
        count = count + 1
    i = i + 1
print(count)                # prints 0 because 'p' != 'P'
    
```

Changing Strings

Strings are **immutable** sets of characters:

- we cannot change any character within a string once it is created

Example:

```
>>> s = "Python Program"
```

```
>>> s[1]
```

```
"y"
```

```
>>> s[1] = "a"
```

```
TypeError: 'str' object does not support item assignment
```

So, how can we update strings?

- (re)assign a variable with a new string
- the new value can be related to the previous value or to a completely different string altogether

```
>>> var1 = "Hello World"
>>> var2 = var1[:6] + "Python"
>>> print(var1)
"Hello World"
>>> print(var2)
"Hello Python"
>>> var1 = var2 + "!"
>>> print(var1)
"Hello Python!"
>>> print(var2)
"Hello Python"
```

Build-in String Methods

Python has a set of built-in methods that you can use on strings.

You can find the full list at

<https://docs.python.org/3/library/stdtypes.html#string-methods>

Warning

All string methods return new values. They do not change the original string.

Let's go through the most popular ones, shall we?

`s.capitalize()` returns a copy of the string with its first character capitalised and the rest lowercased.

```
>>> s = "supercalifragilisticxpialidocious"  
>>> s.capitalize()  
'Supercalifragilisticxpialidocious'
```

```
>>> s = "SupercaliFragilisTicxpialidOciouS"  
>>> s.capitalize()  
'Supercalifragilisticxpialidocious'
```

`# if the string is already capitalised, it has no effect`

```
>>> s = "Supercalifragilisticxpialidocious"  
>>> s.capitalize()  
'Supercalifragilisticxpialidocious'
```

`s.lower()` returns a copy of the string with all the cased characters converted to lowercase.

`s.upper()` returns a copy of the string with all the cased characters converted to uppercase.

```
>>> s = "SUPercalifragiliSTIcexpialidocious"
>>> s.lower()
'supercalifragilisticexpialidocious'
>>> s.upper()
'SUPERCALIFRAGILISTICEXPIALIDOCIOUS'
```

`#` if the string is already lowercased, it has no effect

```
>>> s = "supercalifragilisticexpialidocious"
>>> s.lower()
'supercalifragilisticexpialidocious'
```

`#` if the string is already uppercased, it has no effect

```
>>> s = "SUPERCALIFRAGILISTICEXPIALIDOCIOUS"
>>> s.upper()
'SUPERCALIFRAGILISTICEXPIALIDOCIOUS'
```

`s.count(sub, [start, end])` returns the number of non-overlapping occurrences of substring `sub` in the string `s`. Optionally, you may specify `start` and `end` to look for `sub` in a slice of `s`.

```
>>> s = "supercalifragilisticexpialidocious"
>>> s.count('li')
3
# lowercase vs. uppercase
>>> s.count('LI')
0
# look for 'li' in s[0:10]
>>> s.count('li', 0, 10)
1
>>> s[0:10].count('li')                                     # explicit slicing, same effect
1
```


`s.endswith(sub, [start, end])` returns `True` if the string *ends* with the specified `sub`, otherwise returns `False`.

`s.startswith(sub, [start, end])` returns `True` if the string *starts* with the specified `sub`, otherwise returns `False`.

Use optional parameters **start** and **end** to test a slice of **s**.

```
>>> s = "supercalifragilisticexpialidocious"
>>> s.startswith('super')
True
>>> s.startswith('super', 1, 10)                # s[1:10] -> 'upercalif'
False
>>> s.startswith('uper', 1, 10)
True
>>> s.endswith('ous')
True
>>> s.endswith('calif', 1, 10)
True
```

`s.find(sub, [start, end])` returns the lowest index in the string `s` where substring `sub`. If `sub` is not found, it returns `-1`.

`s.index(sub, [start, end])` works like `s.find()`, but raises `ValueError` when the substring is not found.

Use optional parameters `start` and `end` to test a slice of `s`.

```
>>> s = "supercalifragilisticexpialidocious"
>>> s.find('calif')                    # supercalifragil...
5                                     # s.index('calif') also returns 5
>>> s.find('calio')
-1
>>> s.index('calio')
ValueError: substring not found
>>> 'calif' in s
True
```

Hint

Use `find()` or `index()` if you need the position of the substring. To check if `sub` exists in `s`, you may use the `in` operator.

s.islower() returns **True** if a non-empty string **s** contains only lowercase characters. Otherwise returns **False**.

s.isupper() returns **True** if a non-empty string **s** contains only uppercase characters. Otherwise returns **False**.

s.isalnum() returns **True** if a non-empty string **s** contains only alphanumeric characters². Otherwise returns **False**.

s.isalpha() returns **True** if a non-empty string **s** contains only alphabetic characters³. Otherwise returns **False**.

s.isdecimal() returns **True** if a non-empty string **s** contains only decimal numbers. Otherwise returns **False**.

s.isdigit() returns **True** if a non-empty string **s** contains only digits⁴. Otherwise returns **False**.

s.isnumeric() returns **True** if a non-empty string **s** contains only numeric characters⁵. Otherwise returns **False**.

s.istitle() returns **True** if a non-empty string **s** contains is titlecased. Otherwise returns **False**.

²In short: numbers and letters.

³In short: letters.

⁴In short: decimals, subscripts and superscripts.

⁵In short: decimals, subscripts, superscripts, vulgar fractions, roman numerals, currency numerators.

Acknowledgments

- <https://stackoverflow.com/questions/509211/understanding-slice-notation>